Problem 4 Solution - Agent Harness

Nguyen Tuan Kiet

Approach

Four rules keep the loop safe

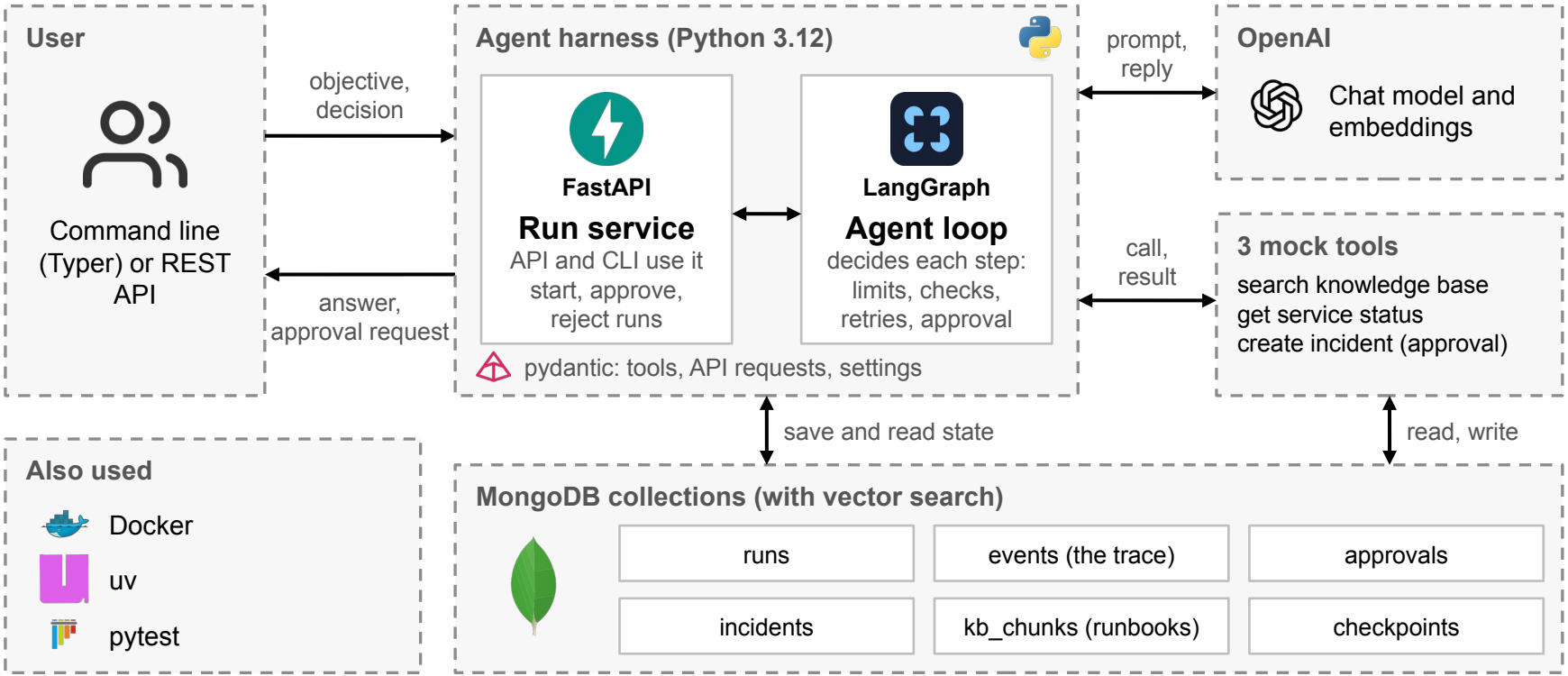
Bounded
Limits on steps, time and retries stop endless loops.

Validated
Every tool call and every result is checked against a schema.

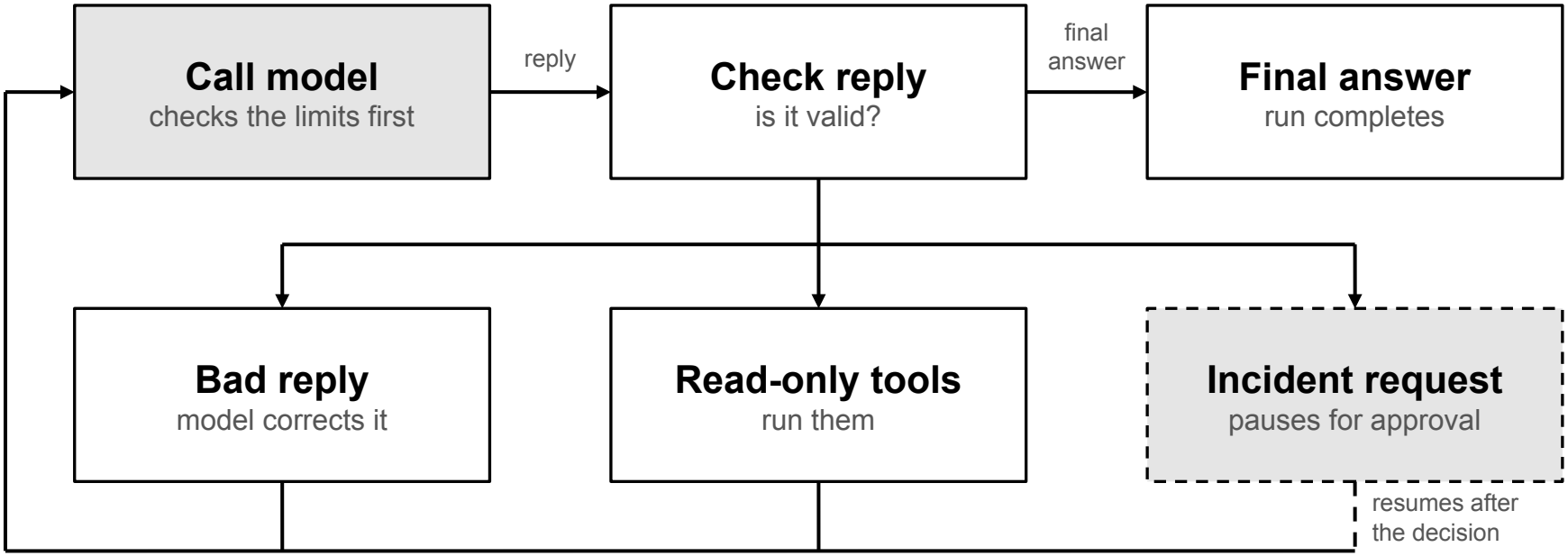
Approved
An incident is created only after a human approves the exact details.

Traceable
Every model call, tool call, error and decision is recorded.

Architecture

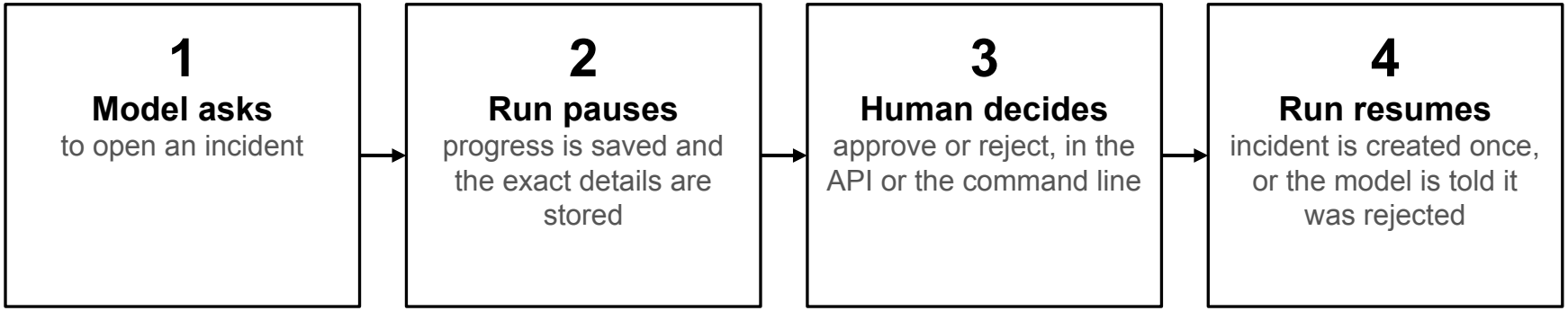


The agent loop



Dashed: the run pauses until a human decides. A limit or an unrecoverable failure also ends the run.

Human approval



Safety rules

- The incident uses the details stored at request time; the approve call carries only the decision
- Approving twice never creates two incidents
- Waiting for a human does not count against the time limit
- A paused run survives a restart

Failures and limits

What goes wrong	What the harness does	Result
Tool fails briefly	Retries a few times, with short waits	Continues if it succeeds
Tool is too slow	Times out, then retries	Error is reported to the model
Tool fails for good or returns bad data	Not retried; result is checked against its schema	Error is reported to the model, which can adapt
Model reply is malformed	Tool is not run; model is asked to correct itself, a limited number of times per run	Run fails if still unusable
Model never stops calling tools	Step and time limits	Run is stopped, history is kept
Unexpected error	Caught and logged	Run is marked failed

Step, time, timeout and retry limits are settings with defaults.

Database design

MongoDB is the only store, including semantic search for the knowledge base.

Collection	What it holds
runs	Each run's objective, status and result
events	The ordered trace of every model call, tool call and decision, also available over the API
approvals	Each approval request and its decision
incidents	Created incidents, at most one per approved request
kb_chunks	Runbook text chunks with vectors for semantic search
checkpoints	Loop state after every step (messages, counters), so a paused run can resume

Every run leaves a full trace, and logs are JSON with the run id.

How it is tested

A scripted fake model replaces OpenAI, so every failure can be repeated.

Unit
Schemas, routing rules, retry and timeout logic

Integration
Full runs against a real MongoDB with the scripted model

API and command line
Start, pause, approve and reject end to end

Scenario	What the test proves
A tool fails twice, then works	It is retried and the run still completes
Bad JSON or an unknown tool from the model	Nothing runs; the model corrects itself or the run fails
The model never stops calling tools	The run stops at its limit and keeps its history
Approved twice, or with changed details	Only one incident is created, from the stored details
Restart while waiting for approval	The approval still works afterwards
A tool result contains instructions	It reaches the model labelled as data, not as instructions

Run it, configure it, call it

Environment variable	Default	Purpose
OPENAI_API_KEY	(required)	OpenAI key; never committed; requests use store=False
OPENAI_MODEL	gpt-4o-mini	Chat model
OPENAI_EMBEDDING_MODEL	text-embedding-3-small	Embeddings for the knowledge base
MONGODB_URI	mongodb://localhost:27017	MongoDB connection
MONGODB_DB	ops_assistant	Database name
MAX_STEPS	8	Max model turns per run
MAX_RUN_SECONDS	60	Max active run time (waiting for approval is not counted)
TOOL_TIMEOUT_SECONDS	5	Timeout per tool attempt
TOOL_MAX_RETRIES	2	Retries for transient errors and timeouts
MAX_REPAIRS	2	Corrections allowed for a malformed model reply
FAILURE_MODE	none	Demo only: transient, permanent, timeout or invalid_output

Run: cp .env.example .env, docker compose up -d, uv sync, uv run python -m src.seed, uv run uvicorn src.main:app (or the CLI: uv run python -m src.cli run "...")

Test: uv run pytest (needs the MongoDB container, never calls OpenAI)

Postman: postman/ops-assistant.postman_collection.json; set base_url, run_id is saved automatically

Limitations and next steps

Limitation	Next step
Single process, single user, no login	Authentication and per-tenant data
A run executes inside the request, so long runs block the caller	Background workers with notifications
Approvals never expire and nobody is notified	Expiry, reminders and an audit of who approved
A crash between approval and resume leaves the run stuck	Automatic recovery of stuck runs
Tools are mocks; search has no reranking	Real integrations, evaluation and better search
Protection against instructions hidden in tool output relies on a prompt rule	Output filtering and a policy check before tools run